

Basi di dati - Appunti mirati all'esame

Sintesi exam-first basata sugli appelli 2024-2025

Giugno 2026

Indice

1. Cosa studiare prima	2
2. Strategia per i 90 minuti	2
Parte I - Modello relazionale, DDL e domande rapide	2
3. Relazione, schema e istanza	2
4. Superchiave, chiave e chiave primaria	2
5. Chiavi esterne e integrità referenziale	3
6. NULL, CHECK e lettura del DDL	3
7. Join esterni: il punto più facile da sbagliare	3
Parte II - SQL: i pattern che coprono quasi tutte le query	4
8. Regola generale	4
9. "Non esiste" / "mai"	4
10. Conteggio includendo lo zero	4
11. Massimo con tutti gli ex aequo	5
12. Più della media	5
13. Coppie e "due righe diverse": self join	5
14. "Tutti": divisione con doppio NOT EXISTS	5
15. Conteggi distinti	6
16. Date	6
Parte III - Algebra relazionale	6
17. Operatori essenziali	6
18. Pattern algebrici d'esame	6
Parte IV - Progettazione ER e traduzione	7
19. Cardinalità da una FK	7
20. Da ER a relazionale	7
21. Generalizzazioni	8
22. Da relazionale a ER: minimizzazione	8
Parte V - Normalizzazione	9
23. Dipendenze funzionali e chiusura	9
24. Forme normali	9
25. Decomposizione senza perdita e conservazione	9
Parte VI - Sicurezza	10
26. Politiche di accesso	10
27. GRANT, grant option e REVOKE	10
28. Cavallo di Troia	10
Parte VII - Transazioni e indici	10
29. Transazioni e ACID	10
30. Strutture fisiche e indici	11

Parte VIII - Ripasso minimo degli argomenti non comparsi	11
31. DBMS, livelli e indipendenza	11
32. Trigger e viste	11
33. NoSQL in una frase	11

Checklist finale da memorizzare **11**

Questi appunti sono costruiti sui cinque appelli disponibili (16/07/2024, 14/01/2025, 10/06/2025, 01/07/2025, 22/07/2025). Il programma del corso è usato per chiarire e completare ciò che compare nelle prove, non per assegnare la stessa importanza a ogni argomento.

Per gli svolgimenti completi si veda [SOLUZIONI_APPELLI.md](#).

1. Cosa studiare prima

Argomento	Presenza nei 5 appelli	Priorità
SQL (join, negazione, aggregazione, massimo, divisione)	5/5	massima
Algebra relazionale	5/5	massima
ER, traduzione e reverse engineering	5/5	massima
Vincoli SQL, chiavi, NULL, effetti delle modifiche	5/5	massima
Normalizzazione	5/5	alta
Sicurezza, GRANT/REVOKE	5/5	alta
Transazioni e proprietà ACID	3/5	media-alta
File e indici	3/5	media-alta
Trigger, viste, PL/pgSQL, NoSQL/MongoDB	0/5	bassa, ripasso finale

Negli appelli estivi 2025 l'esercizio 1 vale 8 punti ed è una **soglia**: se non si raggiungono tutti gli 8 punti, il resto può non essere corretto. Va quindi preparato come una batteria di risposte brevi e molto precise.

2. Strategia per i 90 minuti

1. Leggere DDL, chiavi, NOT NULL, azioni ON DELETE/UPDATE e annotare le FK.
2. Svolgere con calma l'esercizio-soglia; controllare soprattutto NULL, join esterni, duplicati e vincoli impliciti della chiave primaria.
3. Fare prima le query SQL riconducibili a un pattern noto.
4. Scrivere l'algebra con relazioni intermedie e ridenominazioni esplicite.
5. Disegnare ER/logico e teoria breve con definizioni, non con esempi vaghi.
6. Lasciare almeno 8-10 minuti per verificare alias, GROUP BY, casi a zero e direzione delle chiavi esterne.

Parte I - Modello relazionale, DDL e domande rapide

3. Relazione, schema e istanza

- Una relazione è un insieme di tuple su domini atomici.
- **Grado**: numero di attributi.
- **Cardinalità**: numero di tuple dell'istanza.
- L'ordine di righe e colonne non ha significato nel modello relazionale.
- Nel modello relazionale puro non esistono tuple duplicate; SQL, invece, usa normalmente semantica a multinsieme e può produrre duplicati.

4. Superchiave, chiave e chiave primaria

- **Superchiave**: insieme di attributi che identifica univocamente ogni tupla.
- **Chiave candidata**: superchiave minimale. Togliendo un attributo perde l'unicità.

- **Chiave primaria:** una chiave candidata scelta come identificatore principale. È unica per tabella e implica UNIQUE e NOT NULL.
- Una superchiave non minimale si ottiene aggiungendo attributi a una chiave: se code è PK, {code, air_company} è una superchiave ma non una chiave.
- **Attributo primo:** attributo che appartiene ad almeno una chiave candidata.

Attenzione: UNIQUE non equivale sempre a PRIMARY KEY, perché in SQL può ammettere valori nulli; la PK non li ammette.

5. Chiavi esterne e integrità referenziale

Una FK R.X REFERENCES S.Y impone che ogni valore non nullo di R.X compaia nel campo candidato referenziato S.Y.

- La FK è nullable se non ha NOT NULL e non appartiene a una PK.
- Gli attributi di una PK composta sono tutti implicitamente NOT NULL.
- Se non è specificata un'azione, ON DELETE/UPDATE è normalmente NO ACTION: l'operazione che spezzerebbe i riferimenti viene rifiutata.
- CASCADE: propaga eliminazione o aggiornamento alle tuple referenzianti.
- SET NULL: mette a NULL la FK; è possibile solo se la colonna è nullable.

Un'istruzione è atomica. Se un DELETE attiva un CASCADE su una tabella ma viola una FK NO ACTION su un'altra, fallisce **l'intera istruzione**: non resta neppure la cancellazione in cascata parziale.

6. NULL, CHECK e lettura del DDL

- NULL significa valore mancante; NULL = NULL non è vero ma UNKNOWN.
- Si testa con IS NULL / IS NOT NULL, mai con = NULL.
- Una condizione WHERE conserva solo le righe per cui il predicato è TRUE.
- Un CHECK è violato solo quando è FALSE; TRUE e UNKNOWN passano. Per vietare il nullo serve anche NOT NULL.
- Una stringa vuota '' non è NULL in PostgreSQL.
- Non inventare vincoli semantici non presenti nel DDL. Per esempio, expiry_date < release_date è sospetto, ma non viola il DDL se manca il relativo CHECK.

Checklist rapida su un'istanza:

1. tipo e dominio;
2. NOT NULL e PK;
3. unicità di PK/UNIQUE;
4. CHECK;
5. FK, ma solo se è visibile anche la tabella referenziata.

7. Join esterni: il punto più facile da sbagliare

```
FROM R LEFT JOIN S
  ON condizione_di_join AND filtro_su_S
```

Il filtro nell'ON decide quali righe di S si abbinano; non elimina le righe di R, che sono preservate dal LEFT JOIN.

```
FROM R LEFT JOIN S ON ...
WHERE S.id IS NULL
```

è un **anti-join**: conserva le righe di R senza corrispondenza.

Se una riga di R trova due righe di S, SELECT R.* restituisce due copie identiche sotto la semantica SQL, a meno di usare DISTINCT.

Parte II - SQL: i pattern che coprono quasi tutte le query

8. Regola generale

Prima di scrivere SQL, determinare:

- una riga del risultato rappresenta che cosa?
- quali tabelle servono per filtrare e quali per mostrare attributi?
- vanno conservati anche gli oggetti con conteggio zero?
- si cercano uno, tutti i massimi, oppure almeno un caso?
- il risultato può contenere duplicati?

Negli esempi seguenti R è l'entità principale e S contiene associazioni.

9. “Non esiste” / “mai”

Pattern consigliato:

```
SELECT r.*
FROM R AS r
WHERE NOT EXISTS (
    SELECT 1
    FROM S AS s
    WHERE s.rid = r.id
        AND <condizione vietata>
);
```

Include anche gli oggetti senza alcuna riga in S. È il pattern di:

- clienti che non hanno ordini in spedizione;
- clienti che non hanno mai preso libri romantici;
- libri senza posizione;
- persone senza documenti validi.

Alternativa anti-join:

```
SELECT r.*
FROM R AS r
LEFT JOIN S AS s
    ON s.rid = r.id AND <condizione vietata>
WHERE s.rid IS NULL;
```

Evitare NOT IN se la sottoquery può contenere NULL: un solo nullo può rendere il confronto UNKNOWN per tutte le righe.

10. Conteggio includendo lo zero

Il filtro sulla tabella destra deve stare nell'ON:

```
SELECT r.id, COUNT(s.id) AS n
FROM R AS r
LEFT JOIN S AS s
    ON s.rid = r.id
    AND <condizione su S>
GROUP BY r.id;
```

Usare COUNT(s.id), non COUNT(*): la riga esterna prodotta dal LEFT JOIN farebbe risultare 1 anche quando S manca.

Per una somma:

```
COALESCE(SUM(d.quantity * p.price), 0) AS totale
```

COALESCE trasforma la somma nulla degli oggetti senza dettagli in zero.

11. Massimo con tutti gli ex aequo

```
WITH conteggi AS (  
    SELECT r.id, COUNT(s.id) AS n  
    FROM R AS r  
    LEFT JOIN S AS s ON s.rid = r.id  
    GROUP BY r.id  
)  
SELECT *  
FROM conteggi  
WHERE n = (SELECT MAX(n) FROM conteggi);
```

Non usare ORDER BY ... LIMIT 1 se la traccia dice “la persona (o persone)” o se gli ex aequo devono essere restituiti.

12. Più della media

Prima costruire una riga di conteggio per ogni oggetto, poi calcolare la media:

```
WITH conteggi AS (  
    SELECT r.id, COUNT(s.id) AS n  
    FROM R AS r  
    LEFT JOIN S AS s  
        ON s.rid = r.id  
        AND <intervallo temporale>  
    GROUP BY r.id  
)  
SELECT *  
FROM conteggi  
WHERE n > (SELECT AVG(n) FROM conteggi);
```

Specificare se la media è su tutti gli oggetti o solo su quelli “attivi”. La prima interpretazione richiede il LEFT JOIN; la seconda può raggruppare solo S.

13. Coppie e “due righe diverse”: self join

```
SELECT DISTINCT x1.id, x2.id  
FROM X AS x1  
JOIN X AS x2  
    ON <proprietà comune>  
    AND x1.id < x2.id;
```

$x1.id < x2.id$ elimina sia la coppia (x,x) sia il duplicato simmetrico (y,x) . Se le due righe hanno ruoli temporali diversi, è più naturale usare $x2.year = x1.year + 1$.

Pattern d'esame:

- passeggeri sullo stesso volo;
- stesso prodotto e stessa quantità in due ordini distinti;
- stesso premio in anni consecutivi e film differenti;
- data di inizio di un prestito uguale alla fine di un altro.

14. “Tutti”: divisione con doppio NOT EXISTS

“Trova gli A associati a **ogni** B disponibile” significa:

non esiste un B per il quale non esiste l'associazione A-B.

```
SELECT a.*  
FROM A AS a  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM B AS b  
    WHERE NOT EXISTS (  
        SELECT 1
```

```

FROM AB AS x
WHERE x.aid = a.id
      AND x.bid = b.id
)
);

```

È il pattern per persone che hanno richiesto tutti i certificati e artisti che hanno vinto in ogni categoria.

Alternativa con conteggi, valida se si gestiscono bene duplicati e insieme vuoto:

```
HAVING COUNT(DISTINCT x.bid) = (SELECT COUNT(*) FROM B)
```

15. Conteggi distinti

La parola “diversi/e” di solito richiede DISTINCT dentro l'aggregato:

```

COUNT(DISTINCT certificate)
COUNT(DISTINCT venue)

```

Non confondere SELECT DISTINCT (elimina righe duplicate del risultato) con COUNT(DISTINCT x) (conta valori distinti di x).

16. Date

Per un anno usare un intervallo semiaperto, che funziona bene anche con timestamp:

```

date_col >= DATE '2025-01-01'
AND date_col < DATE '2026-01-01'

```

Per “non scaduto” la traccia del 14/01/2025 definisce esplicitamente la validità come expiry_date >= CURRENT_DATE; non aggiungere altri requisiti se non richiesti.

Parte III - Algebra relazionale

17. Operatori essenziali

- selezione: $\sigma_{\text{condizione}}(R)$ - filtra tuple;
- proiezione: $\pi_{\text{attributi}}(R)$ - sceglie attributi ed elimina duplicati;
- ridenominazione: $\rho_{\text{nuovoNome}}(R)$ o $\rho_{\{\text{nuovo} \leftarrow \text{vecchio}\}}(R)$;
- prodotto: $R \times S$;
- theta/equi-join: $R \bowtie_{\text{condizione}} S$;
- unione $\cup$, intersezione $\cap$, differenza $-$;
- divisione $R(A, B) \div S(B)$ restituisce gli A associati a tutti i B di S.

Unione, intersezione e differenza richiedono compatibilità: stesso grado e domini corrispondenti. Ridenominare gli attributi quando necessario.

18. Pattern algebrici d'esame

Oggetti senza una proprietà vietata

```
 $\pi_{\text{id}}(R) - \pi_{\text{id}}(\sigma_{\text{condizione}}(S))$ 
```

Se la condizione richiede altre tabelle, prima costruire un join ristretto.

Almeno A e almeno B

```
 $\pi_{\text{id}}(\sigma_{\text{tipo}='A'}(X)) \cap \pi_{\text{id}}(\sigma_{\text{tipo}='B'}(X))$ 
```

Non usare $\sigma_{(\text{tipo}='A' \wedge \text{tipo}='B')}(X)$: una singola tupla non può avere due valori diversi nello stesso attributo.

Tutti

$\pi_{\{a,b\}}(AB) \div \pi_b(B)$

Gli attributi divisori devono avere lo stesso nome/dominio; usare p se necessario.

Massimo senza aggregati

Per trovare i valori non dominati:

```
T =  $\sigma_{\text{condizione}}(R)$ 
Piccoli =  $\pi_{\{t1.\text{attributi}\}}(\sigma_{\{t1.\text{valore} < t2.\text{valore}\}}(p_{t1}(T) \times p_{t2}(T))$ 
)
Massimi = T - Piccoli
```

Conserva tutti gli ex aequo.

Ottimizzazione

1. spingere le selezioni vicino alle relazioni di origine;
2. spingere le proiezioni, conservando gli attributi necessari ai join;
3. sostituire prodotti seguiti da selezione con join;
4. per un anti-join, sottrarre prima i soli identificatori.

Esempio “film senza premiere”:

```
Premiere =  $\pi_{\text{fid}}(\sigma_{\{\text{is\_premiere}=\text{true}\}}(\text{screening}))$ 
Senza =  $\pi_{\text{fid}}(\text{film}) - \text{Premiere}$ 
Risultato =  $\pi_{\{\text{fid}, \text{title}\}}(\text{film} \bowtie \text{Senza})$ 
```

Parte IV - Progettazione ER e traduzione

19. Cardinalità da una FK

Data S.fk REFERENCES R(pk):

- ogni tupla di S è associata a **0..1 R** se la FK è nullable;
- ogni tupla di S è associata a **1..1 R** se la FK è NOT NULL;
- una tupla di R può normalmente essere associata a **0..N S**;
- UNIQUE(S.fk) abbassa il massimo sul lato R a 1.

Una FK composta va trattata come un'unità concettuale. Una FK ricorsiva crea un'associazione ricorsiva con ruoli espliciti, per esempio figlio e padre.

20. Da ER a relazionale

Entità

Ogni entità diventa una relazione; l'identificatore diventa PK. Attributi opzionali diventano colonne nullable.

Associazione 1:N

Migrare la PK del lato 1 come FK nel lato N. La FK è nullable se la partecipazione del lato N ha minimo 0.

Associazione 1:1

Migrare la chiave in uno dei lati, preferibilmente in quello con partecipazione totale, aggiungendo UNIQUE alla FK. Se l'associazione ha attributi o si vuole mantenerla autonoma, si può creare una relazione separata.

Associazione N:M o n-aria

Creare una relazione con le FK verso tutte le entità partecipanti. Di norma la PK è la combinazione delle FK. Gli attributi dell'associazione restano qui.

Se la stessa coppia può ricomparire in date diverse, l'associazione è storica: includere, per esempio, `data_inizio` nella PK.

Attributo multivalore

Creare una relazione separata, per esempio:

```
libro(bid, titolo)
genere(bid, genere)      PK(bid, genere), FK bid → libro.bid
```

Entità debole

La PK contiene l'identificatore parziale più la PK dell'entità proprietaria; quest'ultima è anche FK.

21. Generalizzazioni

- **Totale (T)**: ogni istanza del padre appartiene ad almeno un figlio.
- **Parziale (P)**: possono esistere istanze del padre in nessun figlio.
- **Esclusiva (E)**: un'istanza appartiene al massimo a un figlio.
- **Sovrapposta (S)**: può appartenere a più figli.

Per padre $E(k, a)$ e figli $E1(b)$, $E2(c)$:

Strategia A - sola relazione padre

$E(k, a, b^*, c^*, \text{tipo})$

- `tipo` è nullable se la gerarchia è parziale;
- con gerarchia sovrapposta servono più indicatori o un valore per ogni combinazione;
- servono vincoli extra-schema per coerenza tra `tipo` e attributi dei figli.

Strategia B - sole relazioni figlie

$E1(k, a, b)$
 $E2(k, a, c)$

È impossibile per una gerarchia parziale, perché perderebbe le istanze del padre che non appartengono a figli. Con sovrapposizione duplica i dati comuni.

Strategia C - padre e figli

$E(k, a)$
 $E1(k, b) \quad \text{PK/FK } k \rightarrow E.k$
 $E2(k, c) \quad \text{PK/FK } k \rightarrow E.k$

È generale e riduce i nulli, ma richiede join. Totalità ed esclusività spesso richiedono vincoli extra-schema.

22. Da relazionale a ER: minimizzazione

- Una tabella con propria PK e attributi descrittivi diventa normalmente entità.
- Una tabella la cui PK è esattamente l'unione di due o più FK e che non ha attributi propri può essere minimizzata in un'associazione N:M.
- Una tabella con PK propria e FK non identificanti resta entità collegata da associazioni.
- Due FK dalla stessa tabella alla stessa entità sono due associazioni/ruoli distinti, non una sola.

Parte V - Normalizzazione

23. Dipendenze funzionali e chiusura

$X \rightarrow Y$ significa: tuple uguali su X devono essere uguali anche su Y . Per cercare una chiave si calcola la chiusura X^+ , aggiungendo ripetutamente gli attributi determinati dalle DF. X è superchiave se X^+ contiene tutti gli attributi; è chiave se è anche minimale.

Regole utili:

- riflessività: se $Y \subseteq X$, allora $X \rightarrow Y$;
- arricchimento: $X \rightarrow Y$ implica $XZ \rightarrow YZ$;
- transitività: $X \rightarrow Y$ e $Y \rightarrow Z$ implicano $X \rightarrow Z$;
- decomposizione: $X \rightarrow YZ$ implica $X \rightarrow Y$ e $X \rightarrow Z$;
- unione: $X \rightarrow Y$ e $X \rightarrow Z$ implicano $X \rightarrow YZ$.

24. Forme normali

1NF

Tutti gli attributi sono atomici; niente liste o attributi multivalore nella stessa cella.

2NF

Ogni attributo non primo dipende **completamente** da ogni chiave candidata: niente dipendenze da una parte propria di una chiave composta. Una relazione con tutte le chiavi semplici è automaticamente in 2NF.

3NF

Per ogni DF non banale $X \rightarrow A$, almeno una condizione vale:

- X è una superchiave; oppure
- A è primo.

BCNF

Per ogni DF non banale $X \rightarrow A$, X deve essere una superchiave. È più forte della 3NF.

Diagnosi tipica: con PK semplice *bid* e DF *scaffale* $\rightarrow$ *ripiano*, la relazione è in 2NF ma non in 3NF/BCNF: *scaffale* non è superchiave e *ripiano* non è primo.

25. Decomposizione senza perdita e conservazione

Una decomposizione è **senza perdita** se il join delle proiezioni ricostruisce esattamente la relazione originaria, senza tuple spurie. Per una decomposizione binaria $R \rightarrow R_1, R_2$, è senza perdita se l'intersezione $R_1 \cap R_2$ determina funzionalmente tutti gli attributi di R_1 oppure tutti quelli di R_2 .

È **conservativa rispetto alle dipendenze** se tutte le DF originarie possono essere verificate sulle singole relazioni decomposte, senza eseguire join.

La sintesi in 3NF può sempre ottenere perdita nulla e conservazione delle DF. Una decomposizione BCNF può sacrificare la conservazione.

Procedura pratica d'esame:

1. calcolare le chiavi;
2. trovare DF che violano la forma richiesta;
3. creare una relazione per ciascun determinante e i suoi dipendenti;
4. eliminare relazioni contenute in altre;
5. assicurarsi che almeno una relazione contenga una chiave originaria;
6. dichiarare PK e forma normale di ogni relazione.

Parte VI - Sicurezza

26. Politiche di accesso

- **Need-to-know / minimo privilegio:** accesso solo ai dati strettamente necessari. Ottima protezione, rischio di negare accessi innocui. Tipica di un sistema chiuso: tutto è vietato salvo autorizzazione esplicita.
- **Maximized sharing / massima condivisione:** massimizza gli accessi, proteggendo solo ciò che è riservato. Più flessibile, meno prudente. Tipica di un sistema aperto: tutto è permesso salvo divieto esplicito.
- **DAC, discrezionale:** proprietari/utenti possono concedere e revocare privilegi. Flessibile, ma non controlla il flusso dopo la lettura.
- **MAC, mandatoria:** soggetti e oggetti hanno classi di sicurezza; le regole centrali controllano anche il flusso. Più rigida e adatta ad alta sicurezza.

System R è discrezionale, chiuso, con amministrazione decentralizzata tramite ownership; supporta controllo per nome e per contenuto.

27. GRANT, grant option e REVOKE

```
GRANT SELECT, INSERT ON R TO utente WITH GRANT OPTION;  
REVOKE SELECT ON R FROM utente CASCADE;
```

- WITH GRANT OPTION permette al destinatario di delegare quel privilegio.
- Si può delegare solo ciò che si possiede con grant option.
- Ogni arco del grafo va etichettato con privilegio, tempo e presenza di GO.
- Conviene disegnare un grafo separato per ogni privilegio.

Revoca ricorsiva: rimuovere l'arco revocato e tutti gli archi che non avrebbero potuto essere creati senza di esso. Un'altra sorgente salva il privilegio solo se è indipendente e, per sostenere una delega già avvenuta, era disponibile con grant option al tempo di quella delega. I timestamp contano.

28. Cavallo di Troia

Un utente autorizzato esegue una procedura apparentemente lecita che contiene codice nascosto. Sfruttando i privilegi dell'esecutore, il codice legge un oggetto segreto e ne copia il contenuto in un oggetto accessibile all'attaccante. Le politiche discrezionali controllano l'accesso iniziale ma non il successivo flusso dell'informazione; una politica mandatoria di controllo del flusso mira a impedirlo.

Parte VII - Transazioni e indici

29. Transazioni e ACID

- COMMIT: termina con successo e rende permanenti gli effetti.
- ROLLBACK: annulla gli effetti non confermati.
- In AUTOCOMMIT ogni istruzione SQL è una transazione.

ACID:

- **Atomicity:** tutto o niente.
- **Consistency:** da uno stato che soddisfa i vincoli a un altro stato che li soddisfa.
- **Isolation:** esecuzione concorrente equivalente a un ordine seriale.
- **Durability:** dopo il commit gli effetti persistono anche dopo guasti.

Anomalie:

- **lost update:** un aggiornamento sovrascrive quello concorrente;
- **dirty read:** T2 legge un dato scritto da T1 prima del commit; T1 poi fa rollback e T2 ha usato un valore mai esistito stabilmente;
- **incorrect summary:** un aggregato legge una parte dei dati prima e una parte dopo un aggiornamento concorrente.

Il dirty read si previene non rendendo visibili modifiche non confermate, con livello almeno READ COMMITTED e/o lock appropriati.

30. Strutture fisiche e indici

- Strutture primarie: sequenziale, hash (accesso calcolato), albero.
- Hash: molto efficiente per uguaglianza, non per intervalli; gestisce collisioni. Nel lessico delle dispense è una struttura primaria.
- Un indice primario determina la disposizione dei record ed è **sparso**: una voce per blocco dati.
- Un indice secondario non determina la disposizione ed è sempre **denso**: deve rappresentare ogni valore/record indicizzato.

Formule:

```
bfr = floor(B / R)
blocchi_dati = ceil(numero_record / bfr)
voci_indice_primario = blocchi_dati
```

Esempio d'esame: 20.000 record, $bfr = 10 \rightarrow 2.000$ blocchi $\rightarrow 2.000$ voci nell'indice primario.

Parte VIII - Ripasso minimo degli argomenti non comparsi

31. DBMS, livelli e indipendenza

Un DBMS gestisce dati persistenti, condivisi e consistenti, offrendo controllo di concorrenza, sicurezza, recupero e linguaggi di interrogazione. I livelli esterno, concettuale e interno separano viste utente, schema logico e memorizzazione fisica. L'indipendenza logica limita l'impatto dei cambiamenti concettuali sulle viste; quella fisica limita l'impatto dei cambiamenti di memorizzazione sullo schema logico.

32. Trigger e viste

Un trigger segue il paradigma evento-condizione-azione e reagisce a INSERT, UPDATE o DELETE, BEFORE/AFTER, per riga o per istruzione. È utile per vincoli non esprimibili nel DDL e dati derivati, ma va evitato quando un vincolo dichiarativo basta.

Una vista è una query con nome. Una vista semplice su una sola tabella può essere aggiornabile; join, aggregazioni e raggruppamenti in genere impediscono l'aggiornamento diretto. WITH CHECK OPTION impedisce modifiche che rendano la riga invisibile nella vista.

33. NoSQL in una frase

I sistemi NoSQL rinunciano a parte dell'uniformità del modello relazionale per flessibilità, distribuzione o specifici pattern di accesso. Nei document store come MongoDB i dati sono documenti JSON/BSON annidati; la scelta tra embedding e riferimenti dipende da cardinalità, aggiornamenti e letture congiunte.

Checklist finale da memorizzare

- PK = UNIQUE + NOT NULL; ogni colonna della PK composta è non nulla.
- FK senza NOT NULL può essere nulla; FK senza CASCADE blocca la modifica.
- LEFT JOIN + WHERE destra.id IS NULL = "senza".
- Filtro sul lato destro nell'ON per non perdere gli zeri.
- COUNT(colonna_destra), non COUNT(*), dopo un LEFT JOIN.
- "diversi" $\rightarrow$ spesso COUNT(DISTINCT ...).
- "tutti" $\rightarrow$ divisione o doppio NOT EXISTS.
- "massimo" $\rightarrow$ confrontare con MAX, conservando gli ex aequo.
- coppie $\rightarrow$ self join; < elimina simmetrie, + 1 esprime anni consecutivi.

- algebra: “almeno A e almeno B” $\rightarrow$ intersezione, non A AND B sulla stessa tupla.
- 2NF elimina dipendenze parziali; 3NF ammette RHS primo; BCNF pretende determinante superchiave.
- revoca: separare i privilegi e rispettare provenienza, grant option e tempo.
- indice primario sparso, secondario denso; $bfr = \text{floor}(B/R)$.
- COMMIT successo; coerenza = rispetto dei vincoli; dirty read = lettura di scritture non confermate.